

# **Auto-Calibration and Optimization Manual for 2x2m UWB Localization System**

UWB Localization & Calibration Package Documentation

June 2026

## **Contents**

# 1 Introduction

Ultra-Wideband (UWB) localization systems based on the Decawave DW1000 transceiver offer high-precision, sub-decimeter ranging capabilities. However, achieving sub-centimeter or highly stable positioning accuracy requires rigorous calibration. Ranging measurements are heavily affected by the internal propagation delays of the transceiver’s RF circuitry and the physical antennas, collectively referred to as the **antenna delay**.

The `2x2mtest2_calibration` package is an integrated hardware-software framework designed to automate the process of:

1. Auto-discovering WiFi-enabled ESP32 UWB anchors.
2. Capturing ranging samples at predefined physical grid coordinates.
3. Performing non-linear least-squares optimization to resolve antenna delay offsets and minor anchor coordinate displacements.
4. Dynamically pushing and persisting calibrated values back to anchor hardware.

# 2 System Architecture & Network Flow

The network relies on UDP broadcasting for telemetry and point-to-point commands, coupled with a central WebSocket interface for dashboard visualization.

Table 1: System Network Port Allocations

| Port | Protocol   | Description                                                                         |
|------|------------|-------------------------------------------------------------------------------------|
| 4210 | UDP        | Tag position and raw ranging broadcasts (Tag $\rightarrow$ Server)                  |
| 4211 | UDP        | Anchor control port for command/response packets (Server $\leftrightarrow$ Anchors) |
| 4213 | UDP        | Anchor discovery beacon broadcasts (Anchors $\rightarrow$ Server)                   |
| 8765 | WebSockets | Communication between the Python server and the HTML GUI                            |

## 2.1 Interaction Loop

1. **Beaconing:** Anchors broadcast discovery packets to port 4213 containing their LSB ID and current delays.
2. **Handshake:** The Python server acknowledges the beacon by sending an **ACK** to the anchor’s IP on port 4211, silencing the beaconing.
3. **Ranging:** The Tag ranges to responders A1, A2, and A3, and broadcasts filtered and raw data to port 4210.
4. **Data Capture:** The user moves the Tag to a grid point and commands a capture via the GUI. The server logs  $N$  packets.
5. **Optimization:** Scipy solves the least-squares problem to compute delay updates.
6. **Command Execution:** Updated delays are sent to the anchors, which update their DW1000 delay registers and store the values in NVS.

### 3 Mathematical Principles

This section documents the equations implemented in the optimization and filtering algorithms.

#### 3.1 The Antenna Delay Ranging Bias

Let  $d_{\text{true}}$  represent the physical Euclidean distance between the Tag and anchor  $i$ . The measured distance  $d_{i,\text{meas}}$  is corrupted by a positive bias  $b_i$  associated with uncalibrated antenna delays:

$$d_{i,\text{meas}} = d_{\text{true}} + b_i \quad (1)$$

The reference clock period of the DW1000 is:

$$T_{\text{ref}} = \frac{1}{499.2 \text{ MHz} \times 128} = \frac{1}{63.8976 \text{ GHz}} \approx 15.65 \text{ ps} \quad (2)$$

Given the speed of light  $c \approx 299,792,458 \text{ m/s}$ , propagation distance per clock cycle is  $d_{\text{unit}} = c \cdot T_{\text{ref}} \approx 4.6917 \text{ mm}$ . In Two-Way Ranging (TWR), the calculated Time of Flight represents the round-trip signal. The programmed antenna delay in the register is subtracted from the timestamps. Increasing the register value by  $\delta_i$  units reduces the measured range by:

$$d_{i,\text{corr}} = d_{i,\text{meas}} - \delta_i \cdot \gamma \quad (3)$$

where  $\gamma = 0.4691 \text{ mm} = 0.4691 \times 10^{-3} \text{ m}$  is the empirical scaling factor.

#### 3.2 Closed-Form Linear Trilateration

Let the 2D positions of the three anchors be  $\vec{a}_1 = (x_1, y_1)^T$ ,  $\vec{a}_2 = (x_2, y_2)^T$ , and  $\vec{a}_3 = (x_3, y_3)^T$ . Given the corrected ranges  $d_1, d_2, d_3$  to the Tag at position  $\vec{p} = (x, y)^T$ :

$$(x - x_i)^2 + (y - y_i)^2 = d_i^2 \implies x^2 + y^2 - 2xx_i - 2yy_i + k_i = d_i^2 \quad (4)$$

where  $k_i = x_i^2 + y_i^2$ .

Subtracting the equation for  $i = 1$  from the equations for  $i = 2$  and  $i = 3$  eliminates the quadratic terms  $x^2 + y^2$ , resulting in a linear system:

$$2x(x_2 - x_1) + 2y(y_2 - y_1) = d_1^2 - d_2^2 + k_2 - k_1 \quad (5)$$

$$2x(x_3 - x_1) + 2y(y_3 - y_1) = d_1^2 - d_3^2 + k_3 - k_1 \quad (6)$$

Representing this system as  $\mathbf{A}\vec{p} = \frac{1}{2}\mathbf{b}$ :

$$\mathbf{A} = \begin{bmatrix} x_2 - x_1 & y_2 - y_1 \\ x_3 - x_1 & y_3 - y_1 \end{bmatrix}, \quad \mathbf{b} = \begin{bmatrix} d_1^2 - d_2^2 + k_2 - k_1 \\ d_1^2 - d_3^2 + k_3 - k_1 \end{bmatrix} \quad (7)$$

The closed-form estimate is resolved as:

$$\vec{p} = \frac{1}{2}\mathbf{A}^{-1}\mathbf{b} \quad (8)$$

Pre-computing  $\mathbf{A}^{-1}$  and  $\vec{k}$  makes this step computationally trivial.

### 3.3 Non-Linear Least-Squares Optimization

Let  $M$  be the number of calibration targets. At each point  $j \in \{1, \dots, M\}$ , the tag is located at a known ground-truth position  $\vec{p}_{\text{true},j}$ . We capture range samples to obtain mean ranges  $\bar{d}_{1,j}, \bar{d}_{2,j}, \bar{d}_{3,j}$ .

For a given correction vector  $\vec{\delta} = (\delta_1, \delta_2, \delta_3)^T$ :

$$d_{i,j,\text{corr}}(\vec{\delta}) = \bar{d}_{i,j} - \delta_i \cdot \gamma \quad (9)$$

The solver computes estimated coordinates  $\vec{p}_{\text{est},j}(\vec{\delta})$  using these corrected values. The objective function is:

$$\min_{\vec{\delta}} \sum_{j=1}^M \|\vec{p}_{\text{est},j}(\vec{\delta}) - \vec{p}_{\text{true},j}\|^2 \quad (10)$$

subject to the bounds  $-\Delta_{\text{bound}} \leq \delta_i \leq \Delta_{\text{bound}}$  ( $\Delta_{\text{bound}} = 500$ ).

When co-optimizing anchor coordinates, the anchor positions are modified as  $\vec{a}_i = \vec{a}_{i,\text{base}} + \Delta \vec{a}_i$  with bounds  $-\Delta_{\text{coord}} \leq \Delta x_i, \Delta y_i \leq \Delta_{\text{coord}}$  ( $\Delta_{\text{coord}} = 0.10$  m). This requires recomputing  $\mathbf{A}^{-1}$  at every optimization iteration.

## 4 ESP32 Tag Filtering Pipeline

To ensure stability during calibration and live tracking, raw ranges are filtered through three sequential layers.

### 4.1 Layer 1: Sliding-Window Median Filter

A window of size  $N = 7$  holds the latest samples. Let the range sequence be  $S = \{s_1, \dots, s_7\}$ . Sorting the sequence yields  $S_{\text{sorted}}$ . The output is:

$$m = S_{\text{sorted}}[3] \quad (11)$$

This rejects transient multipath spikes and high-frequency outliers.

### 4.2 Layer 2: EMA and Outlier Gate

The median output  $m_k$  is passed through a threshold gate relative to the previous filtered estimate  $v_{k-1}$ :

$$v_k = \begin{cases} \alpha \cdot m_k + (1 - \alpha) \cdot v_{k-1} & \text{if } |m_k - v_{k-1}| \leq 0.5 \text{ m} \\ v_{k-1} & \text{if } |m_k - v_{k-1}| > 0.5 \text{ m} \end{cases} \quad (12)$$

where the smoothing factor is set to  $\alpha = 0.25$ . This ensures sudden large jumps are rejected.

### 4.3 Layer 3: Extended Kalman Filter (EKF)

The EKF tracks a 4D state vector representing 2D position and velocity:

$$\mathbf{x}_k = [x_k, y_k, v_{x,k}, v_{y,k}]^T \quad (13)$$

The continuous process noise matrices are discretized, yielding process noise covariance parameters  $q_p = 0.003$  and  $q_v = 0.050$ .

The measurement input is the 2D position estimated via trilateration  $\mathbf{z}_k = [x_{\text{trilat}}, y_{\text{trilat}}]^T$ . The measurement noise covariance  $R_k$  scales dynamically with the trilateration Root Mean Square Error (RMSE) to penalize contradictory measurements:

$$R_k = R_{\text{base}} (1 + 15.0 \cdot \text{RMSE}^2) \quad (14)$$

where  $R_{\text{base}} = 0.025 \text{ m}^2$ . If the trilateration RMSE exceeds 0.80 m, the measurement is discarded and the EKF performs only prediction.

## 5 Step-by-Step Setup Guide

### 5.1 Hardware Deployments

Mount A1, A2, and A3 at the boundary coordinates forming a 2-meter circumradius equilateral triangle centered at (0, 0):

- **Anchor 1 (A1):** (0.00, 2.00) - Top Center
- **Anchor 2 (A2):** (−1.732, −1.00) - Bottom Left
- **Anchor 3 (A3):** (1.732, −1.00) - Bottom Right

Ensure all anchors are raised to the exact same vertical plane height and are clear of metallic obstructions.

### 5.2 Configuration and Compilation

1. Open Code/config.py and modify the WiFi constants:

```
1 WIFI_SSID = "YOUR_WIFI_NAME"  
2 WIFI_PASS = "YOUR_WIFI_PASSWORD"  
3
```

2. Insert the identical WiFi credentials into the ESP32 files:

- Code/Tag/esp32code\_trilateration\_calib.ino
- Code/anchor/anchor1\_top\_calib.ino
- Code/anchor/anchor2\_bottom\_left\_calib.ino
- Code/anchor/anchor3\_bottom\_right\_calib.ino

3. Compile and flash the anchor and tag firmware using the Arduino IDE.

### 5.3 Running the Software Suite

1. Run the central server:

```
1 pip install websockets numpy scipy  
2 python calibration_server.py  
3
```

2. Double-click Code/gui\_calibration.html to launch the browser control interface.
3. Turn on the anchors. Once they broadcast beacons, the server registers their IP addresses and silences them. The GUI sidebar changes to green.

### 5.4 Data Capture and Optimization Procedure

1. Move the Tag to the first grid point on the floor (e.g. (−1.0, 1.0)).
2. Click that coordinate on the GUI map grid.
3. Click **Capture**. A progress bar shows sample accumulation.
4. Repeat for all remaining grid targets.
5. Click **Optimize**. The GUI plots error vector offsets (red lines) and updates delays.
6. Click **Apply**. Delays are transmitted and written to NVS. The system is now calibrated.